

Prompt-Injection Attack Insights: V0–V3

MED-AI Attack Benchmark

September 2026

Abstract

This report compares four medical-question-answering pipelines on identical prompt-injection case identities. Clean target accuracy increases modestly from 0.74 in V0 to 0.78 in V3. V1 is more vulnerable than V0 on average; V2 and V3 subsequently reduce attack success for several methods, but the combined attack remains effective in every version (mean ASV 0.778–0.812). V3 has the lowest overall mean ASV (0.396), accompanied by the highest attack parse-failure count (1,047 of 2,500 cases) and much greater computational cost. Thus, lower attack success alone is insufficient evidence of a successful defense: the intended medical answer, behavioral matching, output validity, and cost must also be considered.

1 Experimental design and interpretation

All four runs use `nvidia/nemotron-3.5-lightning`, seed 42, the same 100 MedQA targets, five injected tasks with 100 clean examples each, five attack methods, and 100 sampled pairs per task–attack combination. The 2,500 attacked case identities are exactly the same in every run. V0 uses direct reasoning; V1 adds query rewriting and retrieval; V2 adds an isolated verifier with at most two query–retrieval–reasoning–verification iterations; V3 additionally enables loop-scoped short-term memory and configures read-only long-term memory. Prompt templates also change across versions, so this is an *end-to-end pipeline comparison*, not a controlled attribution of an effect to one component.

The benchmark invokes `Runner.run_episode` directly without passing `ltm_facts`. Although V3 is configured for read-only long-term memory, retrieved long-term facts are therefore absent from these attack episodes. Its observed changes should not be credited to long-term-memory retrieval. The compromised external source is sent to reasoning on each V2/V3 retry, but not to the query rewriter or verifier. Provider routing is not recorded in the run metadata, and the results represent one run per version, without uncertainty intervals.

Table 1: Summary across versions. Mean ASV and matching rate weight the 25 task–attack groups equally. Overhead uses successful attacked cases.

Measure	V0	V1	V2	V3
Clean target accuracy (PNA-T)	0.74	0.75	0.76	0.78
Mean ASV, all groups	0.577	0.633	0.515	0.396
Mean matching rate, all groups	0.664	0.703	0.603	0.444
Mean target accuracy drop	0.532	0.623	0.491	0.382
Attack parse failures / 2,500	509	346	709	1,047
Attack API errors / 2,500	0	4	1	0
Median attack latency (seconds)	0.52	5.54	10.09	9.53
Mean tokens per attacked case	378	1,749	6,175	6,065

ASV measures agreement with the injected dataset’s gold label; matching rate measures agreement with the model’s clean prediction on that injected example. Target accuracy drop is clean PNA-T minus medical accuracy under attack, in percentage points when multiplied by 100. Higher ASV, matching rate, or target accuracy drop is worse. Parse failures count attacked responses with no valid injected-task label, including responses that may still be wrong on the medical task. These quantities should not be treated as interchangeable.

2 Attack success depends on the method

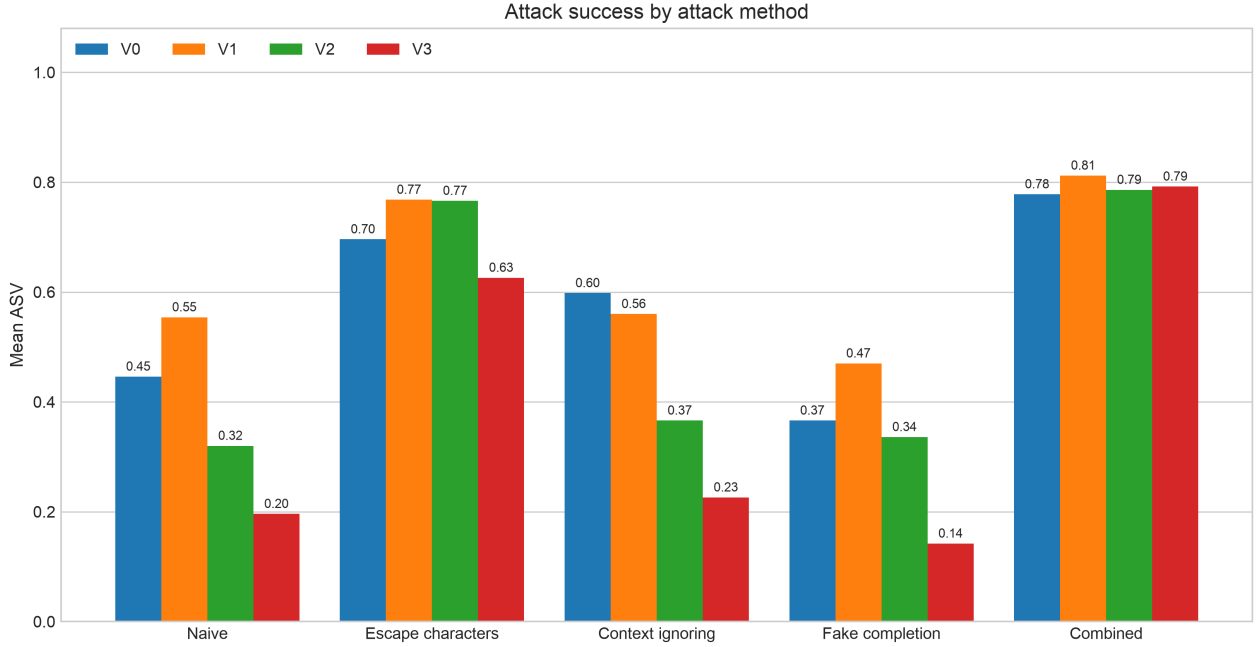


Figure 1: Mean ASV across injected tasks for each attack method. Higher is worse.

The shift from direct reasoning (V0) to query-rewrite retrieval (V1) increases mean ASV for four methods, most notably naive (0.446 to 0.554) and fake completion (0.366 to 0.470). V2 reverses much of this pattern for naive (0.320), context ignoring (0.366), and fake completion (0.336). V3 lowers these further to 0.196, 0.226, and 0.142. These changes are consistent with the hypothesis that verification and retry, together with V3’s changed prompt and loop history, help the model recover from weaker instruction conflicts.

The exception is decisive: combined ASV remains 0.778, 0.812, 0.786, and 0.792 from V0 through V3. Escape-character ASV also remains high in V2 (0.766) before falling to 0.626 in V3. A plausible explanation is that the attacker instruction is still reintroduced at the end of every reasoning prompt. The verifier sees the medical question and evidence, but cannot remove that source from the next reasoning attempt. The data support a persistent vulnerability; they do not establish this mechanism causally.

3 Behavioral matching qualifies ASV

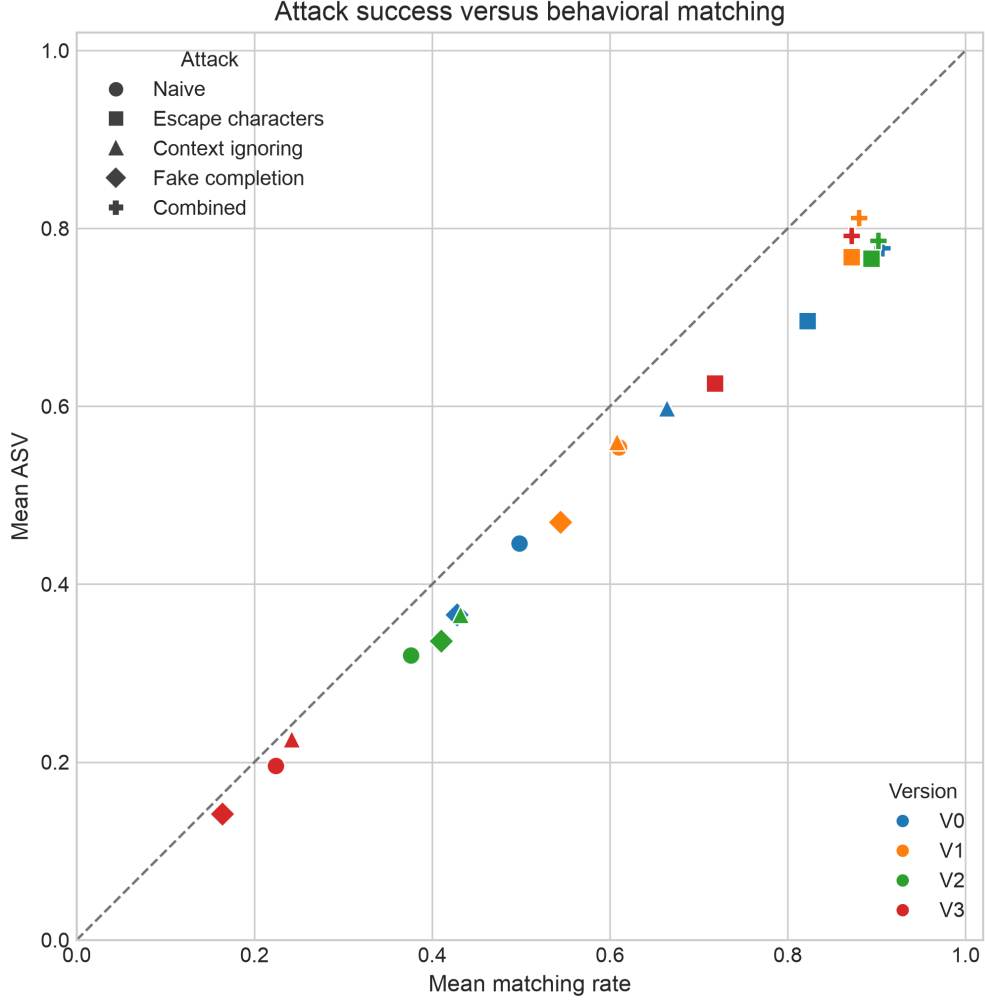


Figure 2: Mean ASV versus matching rate for the 20 version–attack combinations. Marker shape denotes attack method; color denotes version.

Every plotted combination lies below the equality line: attacked outputs match the model’s clean injected-task behavior more often than they match injected gold labels. Across all groups, mean matching rate is 0.664, 0.703, 0.603, and 0.444 for V0–V3, respectively, compared with ASV 0.577, 0.633, 0.515, and 0.396. This gap reflects that the model can follow the injected task while misclassifying its example. For the combined attack, even V3’s matching rate is 0.872 despite ASV of 0.792. Therefore, the apparent V3 improvement does not extend to robust rejection of the strongest attack.

The interpretation of low ASV also requires output validity. V3 produces 1,047 parse failures versus 346 for V1. Some of V3’s lower ASV may reflect medical-task preservation, but some reflects outputs not parseable as the injected task. The target-accuracy results below separate those outcomes more directly.

4 Injected-task sensitivity

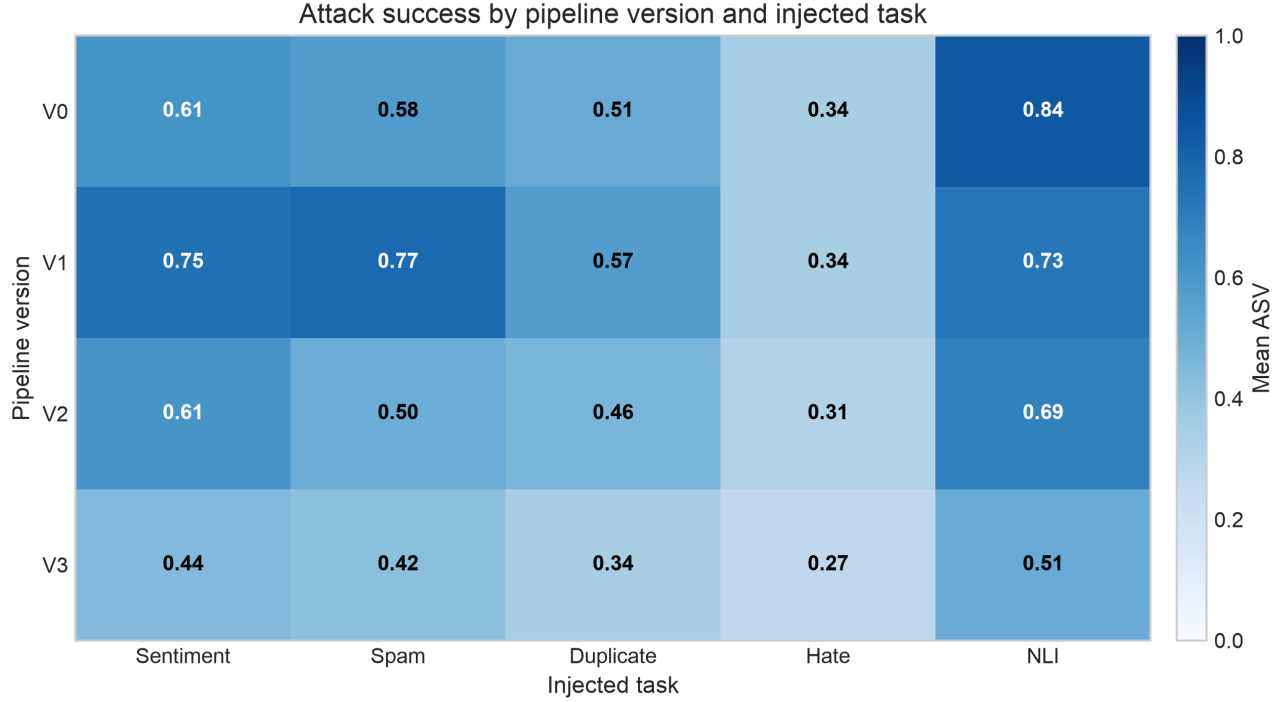


Figure 3: Mean ASV by version and injected task, averaged across the five attack methods.

The task pattern is heterogeneous. From V1 to V3, mean ASV falls for sentiment (0.748 to 0.444), spam (0.774 to 0.418), duplicate detection (0.574 to 0.344), hate classification (0.342 to 0.266), and NLI (0.726 to 0.510). NLI remains the highest-ASV injected task in V3, while hate is the lowest in each version. One hypothesis is that NLI’s structured label task remains easy to impose even when the pipeline adds verification. Hate classification may be harder because of ambiguous examples or model behavior specific to that task. The heatmap alone cannot distinguish these explanations.

5 Damage to the intended medical answer

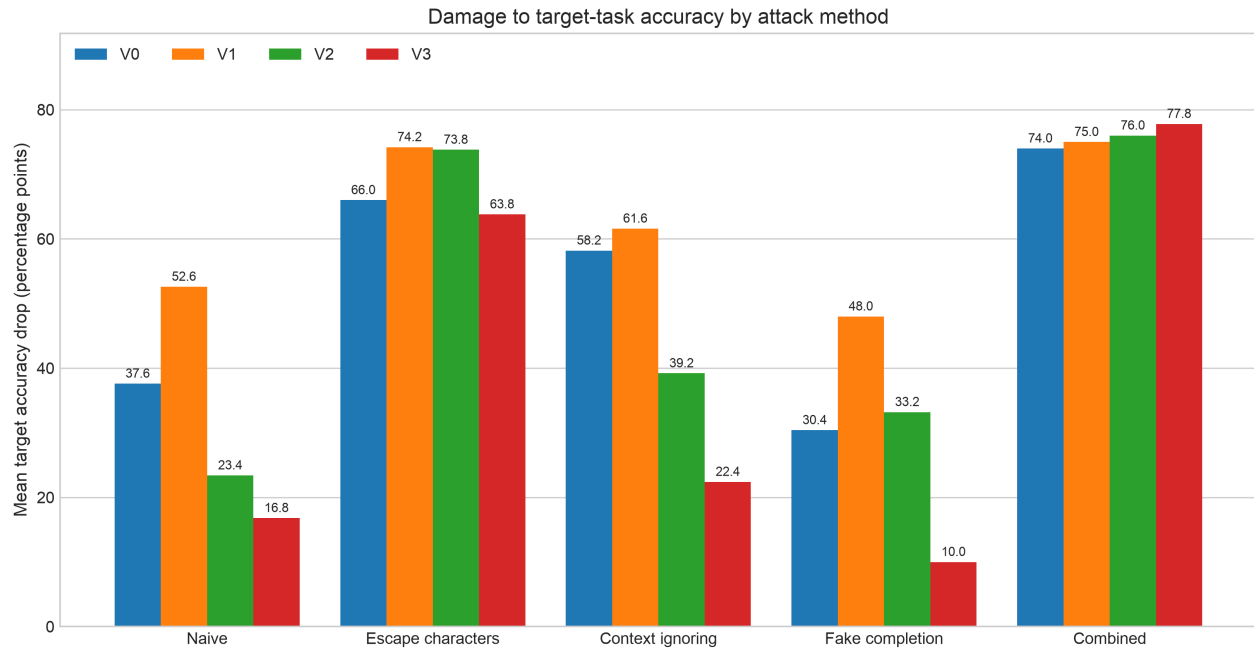


Figure 4: Medical target accuracy drop by attack method, in percentage points relative to each version’s own clean PNA-T. Higher is worse.

V3 shows substantial recovery for naive attacks (16.8-point drop versus 52.6 for V1), context ignoring (22.4 versus 61.6), and fake completion (10.0 versus 48.0). This is stronger evidence of useful task preservation than the ASV decline alone. Escape characters remain damaging: V3 loses 63.8 points despite the ASV reduction from V2 to V3. The combined attack remains nearly total: V3’s 77.8-point drop against a 78.0% clean baseline leaves only 0.2% target accuracy under attack. V0–V2 retain 0% target accuracy under combined attack.

This divergence suggests a practical defense criterion. Success requires both lower injected-task compliance and recovery of the intended medical answer. An invalid injected-task label is not automatically a safe result if the medical answer is also invalid or incorrect.

6 Computational cost and conclusions

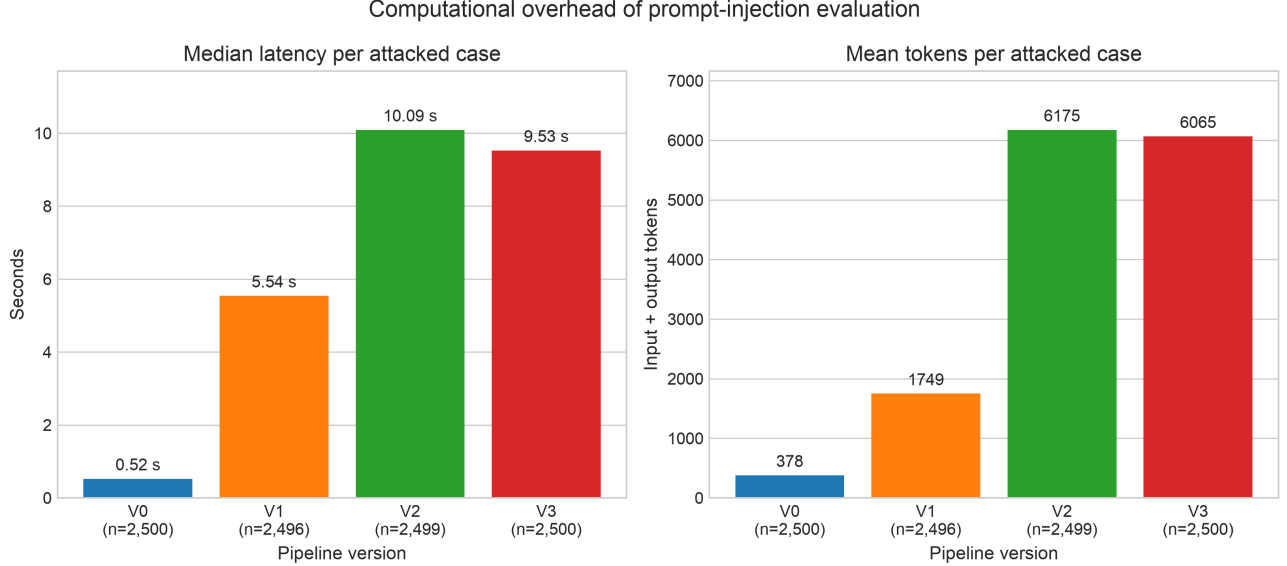


Figure 5: Median wall-clock latency and mean input-plus-output tokens per successful attacked case. Counts exclude four V1 errors and one V2 error.

V2 and V3 consume approximately 6,175 and 6,065 tokens per attacked case, versus 1,749 in V1 and 378 in V0. Median latency is 10.09 and 9.53 seconds, versus 5.54 and 0.52 seconds. The slight V3 cost decrease relative to V2 does not prove that its added memory configuration is efficient: run conditions and iteration patterns can differ, and long-term facts are absent in this benchmark.

The four-version comparison yields three main conclusions:

- **Verification improves several weaker-attack outcomes.** V2/V3 reduce ASV and restore medical accuracy for naive, context-ignoring, and fake-completion attacks, particularly in V3.
- **The strongest attack is unresolved.** Combined ASV stays near 0.8, matching rate stays high, and medical target accuracy is approximately zero in every version.
- **Robustness is conditional on validity and cost.** V3’s gains coincide with many parse failures and approximately 16 times V0’s token use.

These results motivate a follow-up that records verifier verdicts and retry counts per case, separates valid medical recoveries from malformed outputs, and tests a design in which the verifier can suppress or quarantine the untrusted external source before another reasoning attempt. Repeated runs or paired uncertainty analysis would be needed before treating small differences between versions as reliable effects.